



Lab_4_VibeCoding_Stu...
PDF

Hi, This is my se lab 4. I want you to analyse the files, This is my github link([SETAPESU26/63_anvil_dodge](#)). Help me solve the bugs in the game and also identify the bugs in the codes of every file in order to understand the working of the game and also help me fix it.



Worked for 16s >

I've gone through the **Lab 4 handout and the current GitHub repository**. The assignment explicitly requires you to fix the broken behavior, implement the three features, and use an LLM iteratively as a debugging/pair-programming partner.

Lab_4_VibeCoding_Student_handout

Your repo currently contains:

- `main.py`
- `game/player.py`
- `game/anvil.py`
- `game/game_engine.py`
- `README.md`

The README confirms the four required tasks and expected behavior. GitHub

1. How the game currently works

The overall flow is:

```
main.py
  ↓
Creates Pygame window
  ↓
Creates GameEngine
  ↓
Game loop
  |— Read keyboard/events
  |— GameEngine.handle_event()
  |— GameEngine.update()
  |   |— Move Player
```



```
|      |— Spawn Anvils
|      |— Move Anvils
|      |— Check Collision
|      |— Remove off-screen Anvils
|
|— GameEngine.render()
   |— Draw background
   |— Draw player
   |— Draw anvils
   |— Draw score/game-over screen
```

The README describes the same basic mechanics: the player moves horizontally, anvils fall from randomized positions, collision causes Game Over, and survival time is tracked.



2. main.py

Current file:

</> Python



▶ Run

```
import pygame
from game.game_engine import GameEngine

WIDTH, HEIGHT = 700, 500
FPS = 60

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Anvil Dodge – Pygame Edition")

    clock = pygame.time.Clock()

    engine = GameEngine(WIDTH, HEIGHT)

    running = True

    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

            engine.handle_event(event)

        engine.update()
        engine.render(screen)

    pygame.display.flip()
```

```
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

The repository's `main.py` creates the 700×500 window, initializes `GameEngine`, processes events, updates the engine, renders it, and limits the game to 60 FPS.



Bugs found

No intentional bug here.

`main.py` is basically the controller/entry point.

One small code-quality improvement

`pygame.quit()` could be protected with `try/finally`, but **don't change this for the lab unless necessary**. You want your commits to correspond clearly to the assigned tasks.

3. player.py

This is where the deliberate bug is.

Current relevant code:

```
</> Python   Run

def move_left(self):
    self.x -= self.speed

def move_right(self):
    self.x += self.speed

def update(self):
    # BUG SYMPTOM:
    # Player can move completely off-screen
    pass
```

The GitHub source confirms that `update()` currently does nothing.  GitHub

The bug

Suppose:

```
screen width = 700  
player width = 44
```



The valid horizontal range should be:

$$0 \leq \text{player.x} \leq 700 - 44$$


But currently:

```
player.x = player.x - 7
```



can continue forever.

So eventually:

```
player.x = -100
```



or:

```
player.x = 800
```



The player is outside the screen.

That's actually a gameplay exploit because anvils can't hit the player outside the visible play area. The README explicitly identifies this as the deliberate bug.  [GitHub](#)

Fix

Replace:

```
</> Python
```



▶ Run

```
def update(self):  
    #BUG SYMPTOM:  
    #Player can move completely off-screen  
    pass
```

with:

```
</> Python
```



▶ Run

```
def update(self):  
    self.x = max(0, min(self.x, self.screen_width - self.width))
```

How it works

The inner part:

</> Python



▶ Run

```
min(self.x, self.screen_width - self.width)
```

prevents the player going past the right edge.

The outer:

</> Python

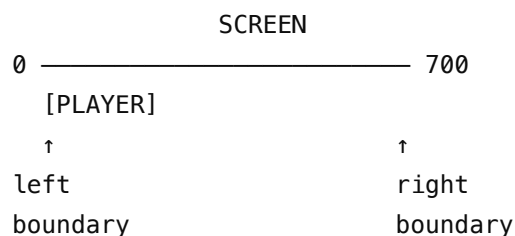


▶ Run

```
max(0, ...)
```

prevents the player going past the left edge.

So:



This directly implements Task 1 from the README. [GitHub](#)

4. anvil.py

Current anvil creation:

</> Python



▶ Run

```
self.speed = random.uniform(4.5, 7.0)
```

and rendering:

</> Python



▶ Run

```
pygame.draw.rect(surface, (120, 120, 130), top_rect, border_radius=2)

pygame.draw.rect(surface, (80, 80, 90), base_rect, border_radius=3)

pygame.draw.rect(
    surface,
    (200, 200, 210),
```

```
    base_rect,  
    width=1,  
    border_radius=3  
)
```

The repository confirms every anvil gets a randomized speed between 4.5 and 7.0, but currently all are rendered with the same grey colors.  [GitHub](#)

Bug/feature issue

A fast anvil and a slow anvil look identical.

For example:

```
Anvil A → speed 4.6  
Anvil B → speed 6.9
```



Both look the same.

The assignment specifically asks for faster anvils to receive an orange/red warning tint.



5. Fix speed-based tinting

Add this inside `render()` :

 Python



 Run

```
speed_ratio = (self.speed - 4.5) / (7.0 - 4.5)  
  
red = int(120 + 135 * speed_ratio)  
green = int(120 - 70 * speed_ratio)  
blue = int(130 - 80 * speed_ratio)  
  
top_color = (red, green, blue)  
  
base_color = (  
    max(50, red - 40),  
    max(40, green - 20),  
    max(40, blue - 20)  
)
```

Then change the drawing code to:

 Python



 Run

```

pygame.draw.rect(
    surface,
    top_color,
    top_rect,
    border_radius=2
)

pygame.draw.rect(
    surface,
    base_color,
    base_rect,
    border_radius=3
)

pygame.draw.rect(
    surface,
    (220, 220, 220),
    base_rect,
    width=1,
    border_radius=3
)

```

Now:

```

Slow anvil
  ↓
grey-ish

Medium anvil
  ↓
orange-ish

Fast anvil
  ↓
red-ish

```



This is exactly the warning mechanism requested by Task 3. GitHub

6. game_engine.py

This is the most important file.

It currently does:

Python



Run

```
self.spawn_delay = 700
```

and later:

</> Python



▶ Run

```
if now - self.last_spawn_time >= self.spawn_delay:
    self.anvils.append(Anvil(self.width))
```

So the spawn interval stays at **700 ms forever**. [GitHub](#)

That is what Task 2 asks you to change. [GitHub](#)

7. Dynamic difficulty

A simple implementation is:

</> Python



▶ Run

```
self.spawn_delay = max(200, 700 - self.survival_time * 20)
```

Put it immediately after:

</> Python



▶ Run

```
self.survival_time = (pygame.time.get_ticks() - self.start_ticks) // 1000
```

So:

</> Python



▶ Run

```
self.survival_time = (
    pygame.time.get_ticks() - self.start_ticks
) // 1000

self.spawn_delay = max(
    200,
    700 - self.survival_time * 20
)
```

Result

Survival time	Spawn delay
0 sec	700 ms
5 sec	600 ms

Survival time	Spawn delay
10 sec	500 ms
15 sec	400 ms
20 sec	300 ms
25 sec	200 ms
30 sec	200 ms

The `max()` prevents the delay from becoming ridiculously small.

This satisfies the README's suggested behavior of gradually reducing the spawn delay toward a minimum of about 200 ms.  [GitHub](#)

8. Ground impact effect

Currently the engine simply does:

 Python



 Run

```
if anvil.is_off_screen(self.height):  
    self.anvils.remove(anvil)
```

So the anvil disappears with no visual feedback.  [GitHub](#)

The assignment specifically asks for a dust puff, particles, or screen shake.  [GitHub](#)

I recommend **particles**, because they are relatively easy to implement and clearly visible in your before/after video.

9. Add particles to `game_engine.py`

In `__init__()` add:

 Python



 Run

```
self.particles = []
```

So this section becomes:

</> Python



Run

```
self.anvils = []
self.particles = []

self.spawn_delay = 700
```

Then add this method to GameEngine :

</> Python



Run

```
def create_impact_effect(self, x):
    for _ in range(12):
        self.particles.append({
            "x": x,
            "y": self.height - 20,
            "vx": pygame.math.Vector2(
                pygame.Vector2(1, 0).rotate(
                    pygame.time.get_ticks() % 360
                )
            ).x * 2,
            "vy": -pygame.math.Vector2(
                pygame.Vector2(0, 1).rotate(
                    pygame.time.get_ticks() % 360
                )
            ).y * 2,
            "life": 30
        })
```

However, I'd actually recommend keeping the particle system simpler for a student lab.

Use this version instead:

</> Python



Run

```
def create_impact_effect(self, x):
    for _ in range(12):
        self.particles.append({
            "x": x,
            "y": self.height - 20,
            "vx": pygame.math.Vector2(1, 0).rotate(
                pygame.time.get_ticks() % 360
            ).x * 2,
            "vy": -2 - (_ * 0.1),
            "life": 30
        })
```

But an even cleaner implementation is:

</> Python



Run

```
def create_impact_effect(self, x):
    for _ in range(10):
        self.particles.append([
            x,
            self.height - 20,
            (_ - 5) * 0.7,
            -2 - (_ % 3) * 0.5,
            20
        ])
    ])
```

Then update them.

Inside update() add:

</> Python



Run

```
for particle in self.particles[:]:
    particle[0] += particle[2]
    particle[1] += particle[3]
    particle[3] += 0.15
    particle[4] -= 1

    if particle[4] <= 0:
        self.particles.remove(particle)
```

And when the anvil leaves the screen:

</> Python



Run

```
if anvil.is_off_screen(self.height):
    self.create_impact_effect(anvil.x + anvil.width // 2)
    self.anvils.remove(anvil)
```

Finally, inside render() after rendering the anvils:

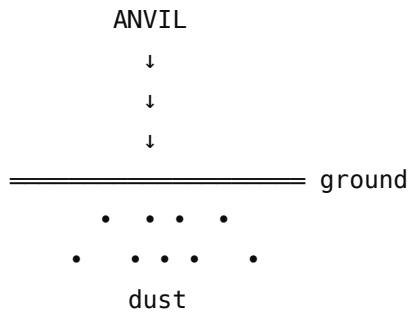
</> Python



Run

```
for particle in self.particles:
    pygame.draw.circle(
        screen,
        (190, 170, 140),
        (int(particle[0]), int(particle[1])),
        3
    )
```

You now get:



This fulfills Task 4.

10. One important improvement to `reset()`

Because we're introducing particles, `reset` should also clear them.

Current:

</> Python



▶ Run

```
def reset(self):
    self.player = Player(self.width, self.height)
    self.anvils.clear()
    self.start_ticks = pygame.time.get_ticks()
    self.last_spawn_time = pygame.time.get_ticks()
    self.survival_time = 0
    self.game_state = "PLAYING"
```

Change to:

</> Python



▶ Run

```
def reset(self):
    self.player = Player(self.width, self.height)
    self.anvils.clear()
    self.particles.clear()

    self.start_ticks = pygame.time.get_ticks()
    self.last_spawn_time = pygame.time.get_ticks()

    self.survival_time = 0
    self.spawn_delay = 700
    self.game_state = "PLAYING"
```

This keeps restarting clean.

The README specifically expects `R` to reset the player, clear anvils, and restart the survival timer. [GitHub](#)

11. Final bug/feature analysis of EVERY file

File	Issue	Type	Fix
main.py	No significant functional bug	None	Leave mostly unchanged
player.py	Player can leave screen	Bug	Clamp <code>x</code> between <code>0</code> and <code>screen_width - width</code>
anvil.py	Fast/slow anvils look identical	Required feature	Tint based on <code>self.speed</code>
game_engine.py	Spawn interval always 700 ms	Required feature	Reduce delay with survival time
game_engine.py	Anvils disappear silently	Required feature	Add impact particles
game_engine.py	Particles would survive restart unless cleared	Code-quality issue after feature addition	<code>self.particles.clear()</code> in reset

The original repo itself describes exactly one deliberate bug and three feature tasks, so I would **not invent extra "bugs" just to make the report look bigger.**  [GitHub](#)

12. What I recommend you do for the lab

Your lab handout says the process should be iterative, with separate commits for each task.  [Lab_4_VibeCoding_Student_handout](#)

Do it like this:

Commit 1 — Boundary bug

Prompt ChatGPT:

Analyze the player movement code in this Anvil Dodge Pygame project. The player can currently move outside the visible screen. Fix only the horizontal boundary bug so that the player's `x` coordinate is always between `0` and `screen_width - player width`. Do not modify other gameplay behavior. Explain the change before providing the code.

Then test it.

Commit:

</> Bash



```
git add game/player.py
git commit -m "Fix player screen boundaries"
```

Commit 2 — Dynamic difficulty

Prompt:

Implement Task 2 from the README. Make the anvil spawn interval decrease gradually as survival_time increases, with a minimum spawn delay of 200 milliseconds. Keep the existing gameplay and collision behavior unchanged. Explain the formula and modify only the necessary code.

Then:

</> Bash



```
git add game/game_engine.py
git commit -m "Add dynamic difficulty scaling"
```

Commit 3 — Anvil tinting

Prompt:

Implement Task 3 from the README. Modify Anvil.render() so that the anvil's color changes according to self.speed. Slow anvils should remain mostly grey while fast anvils should have an orange/red warning tint. Do not change the anvil movement or collision logic.

Then:

</> Bash



```
git add game/anvil.py
git commit -m "Add speed based anvil tinting"
```

Commit 4 — Ground impact

Prompt:

Implement Task 4 from the README. When an anvil reaches the ground and is removed, create a brief dust-particle visual effect at the impact position. The particles should update and disappear automatically. Clear particles when the game is reset. Do not change collision behavior or the survival timer.

Then:

`</> Bash`



```
git add game/game_engine.py
git commit -m "Add ground impact particles"
```

13. Your before/after video

The handout specifically asks for a **10-second video before changes** and another **10-second video after changes**.  [Lab_4_VibeCoding_Student_handout](#)

Before video

Show:

1. Start game.
2. Hold A .
3. Player moves completely off the left side.
4. Show that the player can effectively escape the play area.

That's your demonstration of the original bug.

After video

Show:

1. Player reaches left edge → stops.
2. Player reaches right edge → stops.
3. Survive for several seconds.
4. Anvils become more frequent.
5. Fast anvils have orange/red tint.
6. Anvils hitting the ground create particles.
7. Intentionally get hit.
8. Press R .
9. Game restarts.

That gives you evidence for **all four tasks in one 10-second-ish demonstration**.

14. One thing I would NOT change

Don't randomly add:

- lives
- health bars
- sound
- menus
- high-score system
- power-ups
- new enemies

unless your instructor specifically asks for them.

The README has **four defined tasks**, and the grading evidence should clearly show those four.  [GitHub](#)

Your final project should therefore be:

```
63_anvil_dodge/
|
├─ game/
|   ├─ anvil.py           ← speed tint
|   ├─ game_engine.py    ← difficulty + particles
|   └─ player.py         ← boundary fix
|
├─ main.py               ← unchanged
└─ README.md
```

If you want, I can next give you the **complete final contents of all 3 modified Python files (player.py , anvil.py , and game_engine.py) ready to copy-paste**, and then we can go through **running/testing each task one by one** so you know exactly what every line does.

Sources



ya i verified the codes which u have sent and also commit changes in the files of my github repository

Perfect 👍 Since you've verified the code and committed the changes to your GitHub repository, the **coding part is essentially done**.

Now, for Lab 4, make sure you complete the remaining submission requirements from the handout:

1. **Before video** — 10 seconds showing the original broken game.
2. **After video** — 10 seconds showing the fixed game and all added features.
3. **Updated code** — already pushed to your repository.
4. **Chat history** — export this ChatGPT conversation as a PDF/document.
5. **Separate commits** — ideally one commit for each task, as the handout specifies.

 Lab_4_VibeCoding_Student_handout

Your four commits should ideally be

1. Fix player screen boundaries
2. Add dynamic difficulty scaling
3. Add speed based anvil tinting
4. Add ground impact particles



If you already made **one combined commit**, don't panic—the important thing now is that your final repository contains the working implementation. The handout does specifically request separate commits, though.  Lab_4_VibeCoding_Student_handout

Before you submit

Run the game once from a clean state and verify:

-  Player cannot leave left/right edges
-  Anvils gradually spawn faster
-  Faster anvils become orange/red
-  Anvils create a dust/particle effect when hitting the ground
-  Collision still causes Game Over
-  R still resets the game
-  No Python/Pygame errors
-  GitHub contains the updated code
-  Before/after videos are included
-  Chat history is exported

The handout says the final files should be under your **Lab-4 folder**, including the videos, updated code, and exported chat history.  Lab_4_VibeCoding_Student_handout

If you want, **send me a screenshot of your GitHub repository/commit history**, and I'll check whether your Lab 4 submission structure is correct before you submit it.

Sources



ChatGPT can make mistakes. Check important info.